

Message Authentication:

Secure hash-based constructions

Confidentiality $\neq$ Integrity

→ Confidentiality

→ Message hiding

⇒ Only the **legitimate destination** should be able to **see** the content of a message

→ Integrity

→ Message authenticity

⇒ **Nobody** should be able to **modify** a message while in transmission

⇒ Only the **legitimate source** should be able to **forge** a message

→ Corollary: you **MUST** have secret key (remember WEP CRC ☺)

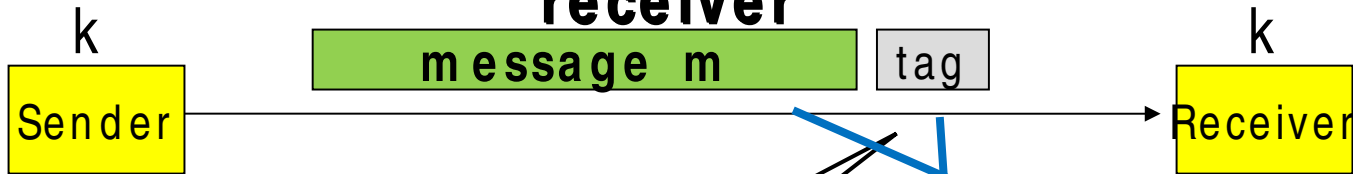
→ Encryption does NOT guarantee any integrity!

⇒ Exception: AEAD ciphers (Authenticated encryption): designed to guarantee both confidentiality and integrity

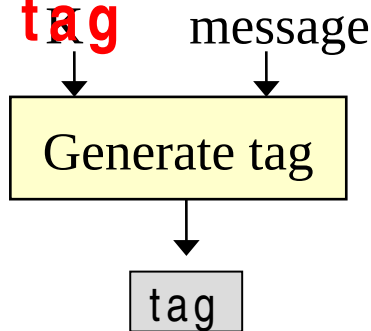
→ Integrity-only: often is the only thing you want

Message authentication (with symmetric key)

Key: secret known by
both sender and
receiver

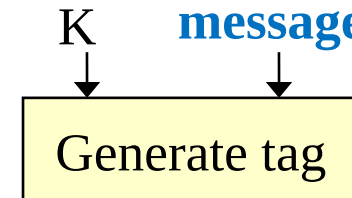


**Generate
tag**



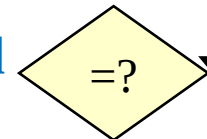
Tag sent
along
with
message

**Verify
Tag**



**Cryptographically strong
function:** Input: K , message
output: short authentication
code

Received
Tag



Y/N = message
OK/NO

Not reversible! (output shorter
than input)

Message authentication: what it does NOT protect?



Protects from MITM attacker

message m

TAG

$$\text{TAG} = F(K, m)$$

Modify
m



But (assumptions on F = cryptographically strong):

1) Sees TAG & m: must NOT be able to «invert» F and get K

2) Cannot change TAG $\rightarrow \text{TAG}^* = F(K, m^*)$ without knowing K

3) Cannot change $m \rightarrow m^*$ such that $F(K, m) = F(K, m^*)$

$$F(K, m^*) = \text{TAG}^* \neq \text{TAG}$$

Someone

modified m !!



Protects from message spoofing

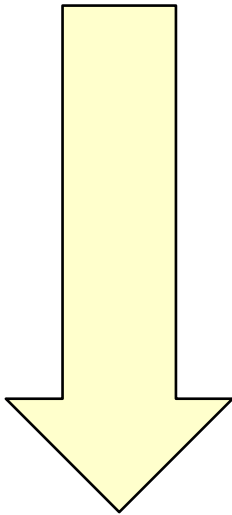


Spoofed message X

TAG = $F(K, X)$

?????

(assumptions on F =
cryptographically strong):
Cannot compute $F(K, X)$ without
knowing K



Spoofed message X

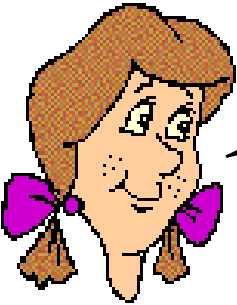
Invalid TAG

$F(K, X) \neq \text{TAG}$

**Spoofed
message!**



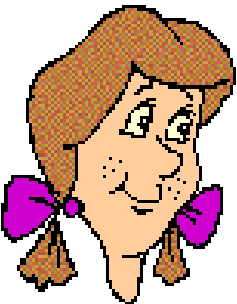
Does NOT protect from replay attacks



I'm sending the same
message twice
as I really want to pay 2000\$

Pay 1000 \$	TAG
-------------	-----

Pay 1000 \$	TAG
-------------	-----

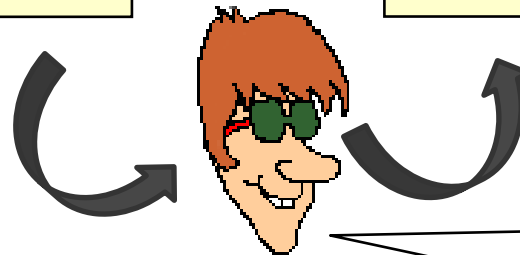


Let's stop. I just
want to pay
\$1000 once.

Pay 1000 \$	TAG
-------------	-----

Ops! This is a
valid
message!

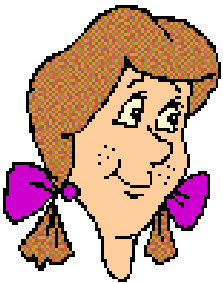
Pay 1000 \$	TAG
-------------	-----



Nah! I'll
make you
pay twice!!

Preventing replay attacks: Nonces

- **Replay attacks prevented if ALL messages are different each other!**
- **How to?**
 - ⇒ Application **MAY** legitimately send two identical messages!



- **Add «nonce» to each packet**
 - ⇒ Nonce=«fresh» information, guaranteed to differ in each message!
 - ⇒ Does not have to remain secret → can be added to the msg in clear
 - Sequence number
 - Random number
 - Time stamp
- **and include it in the TAG computation!**

Test your understanding

- ➔ **Which pros and cons of each type of nonce?**
- ➔ **Sequence number?**
 - ⇒ How to manage «reboots» 😊 ?
- ➔ **Random numbers?**
 - ⇒ How RX can control that packet is a fresh one?
- ➔ **Time stamps**
 - ⇒ Who guarantees time? «now» is really «now»?

MAC = Digital Signature?

→ Digital signature (DS):

⇒ **Nobody** can modify a digitally signed message (except creator)

→ Message Authentication Code (MAC):

⇒ **Nobody except TX and RX (which have the key)** can modify a MAC-authenticated message

→ Both have an identical purpose!

⇒ Message/data Integrity

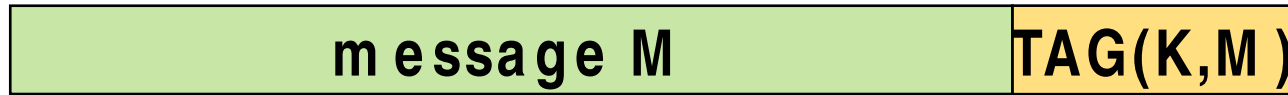
→ But unlike MAC, DS further guarantees non repudiation (source authentication)

⇒ DS = **stronger** form of authentication than MAC

→ But requires different crypto → asymmetric, more later

⇒ Why this is useful? Legal scenarios, Multicast scenarios

Defining Security for Message Authentication Codes



- **Security = Unforgeability**
 - ⇒ Attacker should not be able to create or modify a message
 - ⇒ Implies that attacker should not be able to extract key from pair $\{M, \text{TAG}(K,M)\}$
- **Formally: a game against an attacker model**
 - ⇒ From weaker to stronger: Known Message Attack → Chosen Message Attack → Adaptively Chosen Message Attack
- **Attacker is given (may choose) a number of past message/tag pairs**
 - ⇒ $(m_1, t_1), (m_2, t_2), (m_3, t_3), \dots$
- **Now sees (chooses) message m: must be unable to forge tag**
- **More specifically:**
probability to forge valid pair must be NEGLIGIBLE

Test your understanding

- 1 byte tags
 - No way for attacker to guess tag from msg, beyond pure random choice
 - Is this secure?
-
- **NO! Probability of guessing = $1/256$**
 - Not nearly negligible!!
 - **Note the crucial difference with encryption security definitions!**

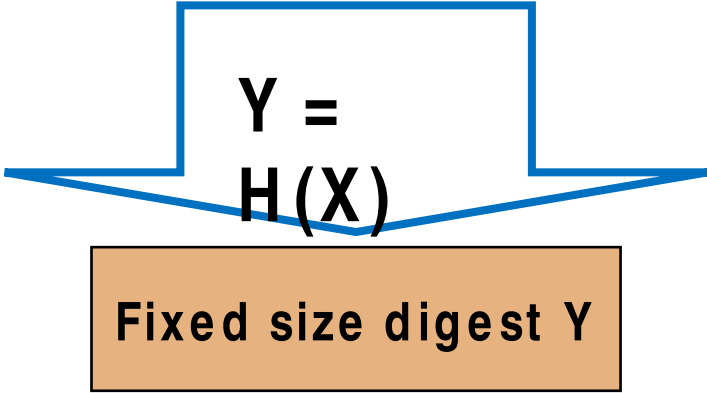
Which (highly non linear) function can we use?

Hash function? What's this?



What is a Hash function?

Any length message X



The diagram illustrates the hash function process. A light green rectangular box at the top contains the text 'Any length message X'. A blue line descends from the center of this box, forming a U-shape. Inside the U-shape, the text 'Y = H(X)' is displayed. The blue line then splits into two diagonal lines that point downwards towards an orange rectangular box at the bottom. This orange box contains the text 'Fixed size digest Y'.

$$Y = H(X)$$

Fixed size digest Y

Arbitrary size $X \rightarrow$ fixed size $Y=H(X)$

(from as short as 1 single bit to... any size) $\rightarrow$ e.g. exactly 256 bits for SHA256

$Y=H(X)$: “fingerprint” (short summary) of X

$H(X)$ should be relatively easy to compute for any given X , making both hardware and software implementations practical

Cryptographic hash functions



Hic ego: laudare igitur eloquentiam et quanta vis sit eius expromere quantamque eis, qui sint eam consecuti, dignitatem afferat, neque propositum nobis est hoc loco neque necessarium **m**. hoc vero sine ulla dubitatione confirmaverim, sive illa arte pariatursive aliqua sive exercitatione quadam sive natura, rem unam esse omnium difficillam. quibus enim ex quinque rebus constare dicitur, earum una quaeque est ars ipsa magna per sese.

quare quinque artium concursus maxumarum quantam vim quantamque difficultatem habeat existimari potest.

H[

1

c6c8258947bff e06ea4a0c8132af33
7a3c74ec81d754a96d5a29e3ca7d8

ce49d

**Fixed size digest
(e.g. SHA-256: 64
hex)**

Giuseppe Bianchi

Hic ego: laudare igitur eloquentiam et quanta vis sit eius expromere quantamque eis, qui sint eam consecuti, dignitatem afferat, neque propositum nobis est hoc loco neque necessarium **s**. hoc vero sine ulla dubitatione confirmaverim, sive illa arte pariatursive aliqua sive exercitatione quadam sive natura, rem unam esse omnium difficillam. quibus enim ex quinque rebus constare dicitur, earum una quaeque est ars ipsa magna per sese.

quare quinque artium concursus maxumarum quantam vim quantamque difficultatem habeat existimari potest.

H[

1

3238ead7fb611463703c47adc4215
aa245a1f1a4a0cea4c11296b466a7

6bbac4

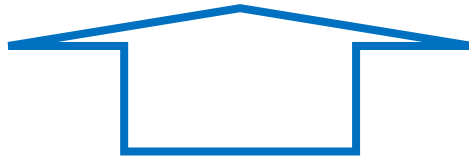
**No way for an attacker to either
create or
purposely modify/extend/replace
initial text**

Property #1 of a cryptographic hash function

→ Preimage resistance (one way)

⇒ Given Y = result of a hash, it is hard to find ANY X such as $H(X)=Y$

? ? ? ?



One way: much more than
«non-invertible»

Infinite (!!) messages can generate
this digest

but you should NOT be able
(in a reasonable time)
to find ANYONE of such
messages!!

Digest:
t:

c6c8258947bffe06ea4a0c8132af33
7a3c74ec81d754a96d5a29e3ca7d8
ce49d

Parenthesis:

Hash functions: computational security

→ **Should resist to brute-force (repeated trials)**

⇒ Consequence: digest cannot be small!

⇒ It rather has to be relatively large (e.g. 256 bit)

Example: 16 bit digest → 2^{16} possible outputs → 65536 values

Assume «random» message → digest mapping (best possible hash function)

Average number of trials to have a success is also

$2^{16} \Rightarrow 65536$ **Thus a large digest is NOT NEARLY ENOUGH!**

Less than 1 ms on my PC (66 Mhash/s)
⇒ Crypto hash functions are very carefully designed to resist to cryptanalysis

→ Example: SHA-1 has 160 bit, but as of today (march 2020) can be broken with only about 2^{61} attempts (practical with today technology)!!

Property #2 of a cryptographic hash function

→ Second preimage resistance (weak collision resistance)

⇒ Given X , it is hard to find another X' such that $H(X) = H(X')$

Good

Message M:
Carriagazzion
potete modificare
questo messaggio se,
come prova di quanto
sto scrivendo,
conservo la sua hash
in una mia penna usb

????
Attacker
cannot find
ANY other
message
which has the
same hash

**Stronger
property than
one-wayness!**

*there are functions
which satisfy
property #1 but fail
property #2*

920d7e8cf22725391c2415ade2635
8f8c099ed37837e1a8911786b0e36

84f23c

Giuseppe Bianchi

Digest:
t:

Test your understanding

(non trivial example)

→ Modular exponentiation

$$\Rightarrow Y = F(x) = g^x \bmod p$$

→ g known value (not necessarily large)

→ p = (very) large prime

$\Rightarrow F(x)$ is one-way, i.e. satisfy property # 1

» In most cases (more later for details on how to choose g and p)

→ Show that it is NOT collision resistant

$\Rightarrow$ i.e., does not satisfy property # 2

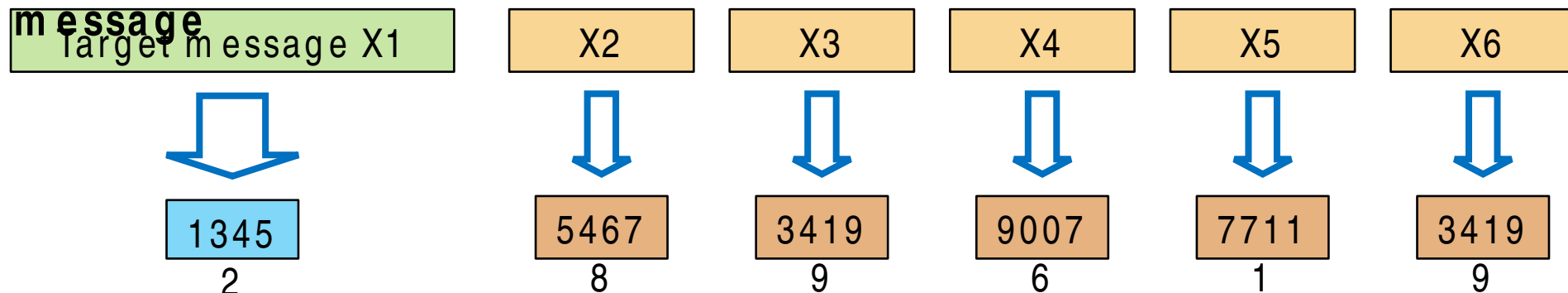
→ Hint: it is a periodic function, and if you find the period... game over! Why? 😊

Property #3 of a cryptographic hash function

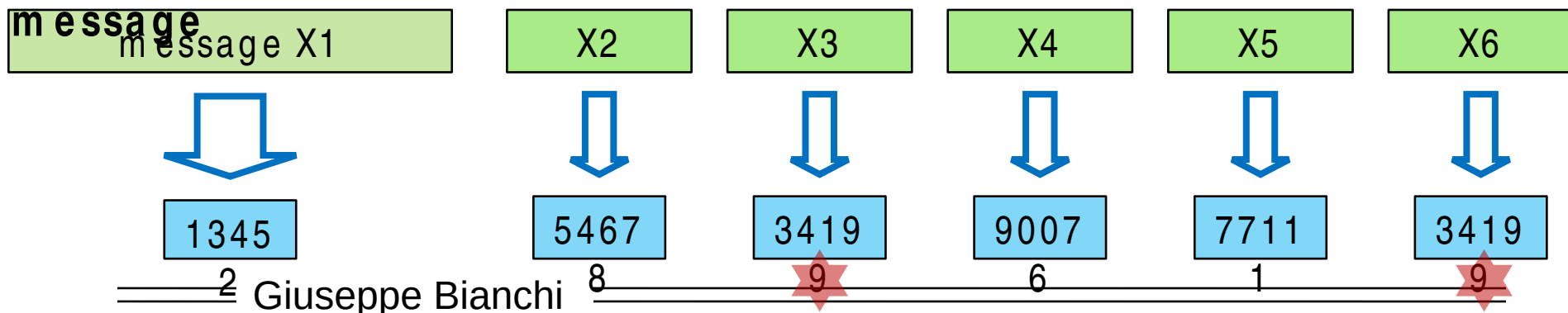
→ Collision resistance (strong collision resistance)

⇒ It is hard to find two generic $X1$ and $X2$ such that $H(X1) = H(X2)$

Weak collision resistance: target



Strong collision resistance: ANY



Birthday paradox

23 people are in a class

(22 + myself)

1. What is the probability that none of you $N=22$ is born in my same day?

Human being X $\rightarrow$ birthday

$$\left(1 - \frac{1}{365}\right)^{22} = \left(\frac{364}{365}\right)^{22} = \dots 94.1\%$$

= you have my same birthday in less than 6% cases

2. What is the probability that no two+ of us $N=23$ are born the same day?

$$\rightarrow \left(1 - \frac{1}{365}\right) \cdot \left(1 - \frac{2}{365}\right) \dots \left(1 - \frac{22}{365}\right) =$$

$$= \frac{365 \cdot (365 - 1) \dots (365 - 22)}{365^{23}} =$$

$$= \frac{365! / (365 - 23)!}{365^{23}} = \dots 49.3\%$$

In more than 50% cases two of us have the same birthday!!!

Not so obvious...

Birthday paradox math

n bit digest →

K = n.

p = collision
probability

N = 2ⁿ

messages

$$\begin{aligned}
 P(\text{no collision}) &= 1 - p = \frac{N! / (N - K)!}{N^K} = \\
 &= \frac{N}{N} \cdot \frac{N - 1}{N} \cdot \frac{N - 2}{N} \dots \frac{N - (K - 1)}{N} = \\
 &= 1 \cdot \left(1 - \frac{1}{N}\right) \cdot \left(1 - \frac{2}{N}\right) \dots \left(1 - \frac{K - 1}{N}\right) = \\
 &= \prod_{i=1}^{K-1} \left(1 - \frac{i}{N}\right) \approx \prod_{i=1}^{K-1} e^{-i/N} = e^{-\sum_{i=1}^{K-1} i / N} \\
 &= e^{-\frac{K(K-1)/2}{N}} \approx e^{-\frac{K^2}{2N}}
 \end{aligned}$$

Remember: for x
small,
1-x approx e^{-x}

Birthday paradox math

So, we concluded that: $1 - p \approx e^{-\frac{K^2}{2N}}$

Let's solve in K: $\log(1 - p) \approx -\frac{K^2}{2N} \Rightarrow K = \sqrt{2N} \cdot \sqrt{\log \frac{1}{1 - p}}$

50% ($p=1/2$) collision probability: $K \approx \sqrt{2} \cdot \sqrt{\log 2} \sqrt{N} \approx 1.177\sqrt{N}$

And since $N=2^n$, using n bits: $K \approx 1.177\sqrt{2^n} \approx 2^{n/2}$

Conclusion: n-bit digest $\rightarrow$ security level is NOT 2^n but $2^{n/2}$

Hash digest size

→ Must be assessed against birthday paradox!

⇒ 32 bits (RAND) → 4.3 billion outputs

→ But 50% collision after $2^{16} \sim 60.000$ msg (very little!)

⇒ 128 bits (MD5) → 50% collision after $2^{64} = 1.8 \times 10^{19}$ (weakish today)

» but MUCH worse than this: MD5 broken via cryptanalysis since 2005

⇒ 256 bit (SHA256) → 50% collision after $2^{128} = 3.4 \times 10^{38}$, OK today

» 3.4×10^{38} →

~ 2000x 4 superenalotto in a row
~ 1 billion centuries of work by whole
worldwide bitcoin network (march 2020 ~ 10^{20} H/s)

Back to Message authentication!

Message Authentication Code using hash functions

ingredient 1: good hash

Good
Messag
e
M

Bad
Messag
e
M'

Good crypto HASH:

Today: SHA-2 family (**SHA256**, SHA224, SHA384, SHA512)

Past: **SHA1, MD5 (both broken)**

next: SHA-3 family (still 224, 256, 384, 512, but new construction)

Most common (used in TLS & Ipsec)

trivial to use (e.g. linux sha256sum command)

(well, apparently ☺ - devil is in details, more later!)



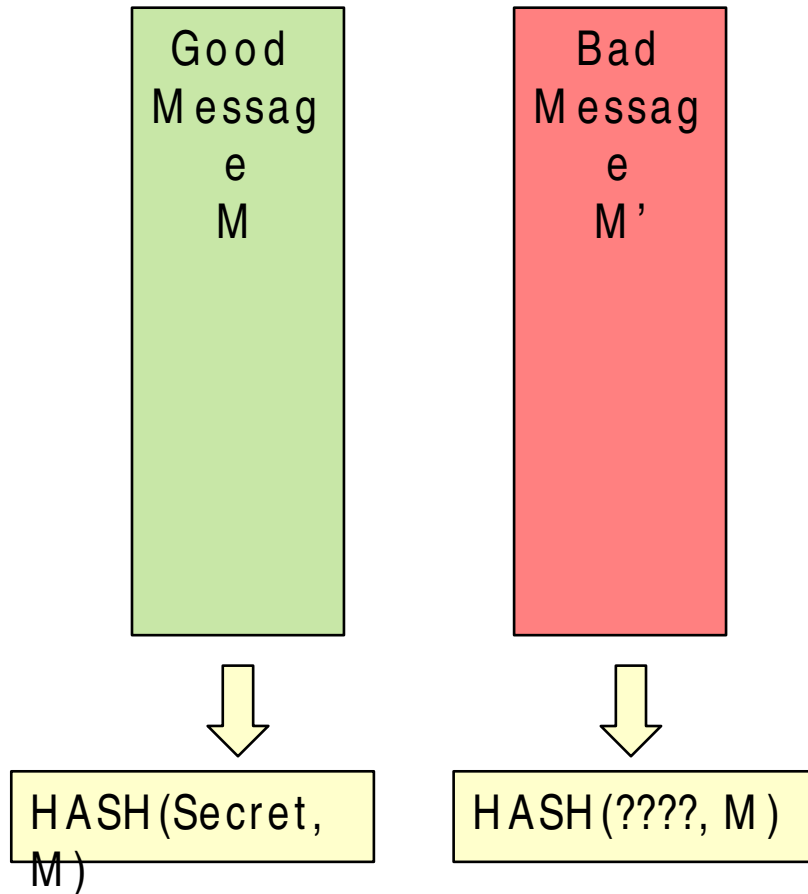
HASH(
M)



VERY HARD to find message M'
whose
Hash is the same of M (collision
resistant hash)

Message Authentication Code using hash functions

ingredient 2: include secret in the hash



**secret =
authentication key**

Since it is not known by
attacker, computationally
hard to construct a valid
message authentication
code

Where to put authentication key in hash?

$H(M ||$

$S) ?$

Dear all, I'd wish
to authenticate
this message. Let
me hash this

0c8132af337a3c74
ec81d754a96d5a29

along with a
secret

3596c18d7c6f7b5df983b1edb
5f7a0d3ddb8b1d041d202a1fc

9b5fb4916d20ed

$H(S ||$

$M) ?$

0c8132af337a3c74
ec81d754a96d5a29

Dear all, I'd wish
to authenticate
this message. Let
me hash this

along with a
secret

12d6aeadec831f55eb5b0556a
0f7e52ce241451ad2556aa70c

04e7f32ee4b19

Auth
key K

Message
M

Tag

Some other fancy way? E.g. «envelope»:

$H(K || M || K) ?$

But why you ask? Silly question, why
should we care?

We would NOT care IF hash function were a «perfect» one: Random Oracle

→ **«oracle» = black box model**

→ **Random oracle:**

⇒ Given any distinct input X , $H(X)$ = **truly**
random value

→ But with repeatability property:
same input X , same output $H(X)$

→ **Fact: NO practical hash function can be
a random oracle**

⇒ Digest cannot be a «truly» random value

But WE HAVE TO BE CAREFUL!
devil is in details...

**→ Matter is: Hash functions are not
black boxes!**

Hash constructions: iterative

Merkle-Damgård construction – example: SHA-256

but all practical hash functions of yesterday (e.g. MD5, SHA1) and today use this – only SHA-3 differs ☺

Length
K mod
 2^{64}

SHA256 IV:

$H_0^{(0)}$:

6a09e667

$H_0^{(1)}$:

bb67ae85

$H_0^{(2)}$:

3c6ef372

$H_0^{(3)}$:

a54ff53a

$H_0^{(4)}$: Initialization

510e527f

$H_0^{(5)}$: Vector

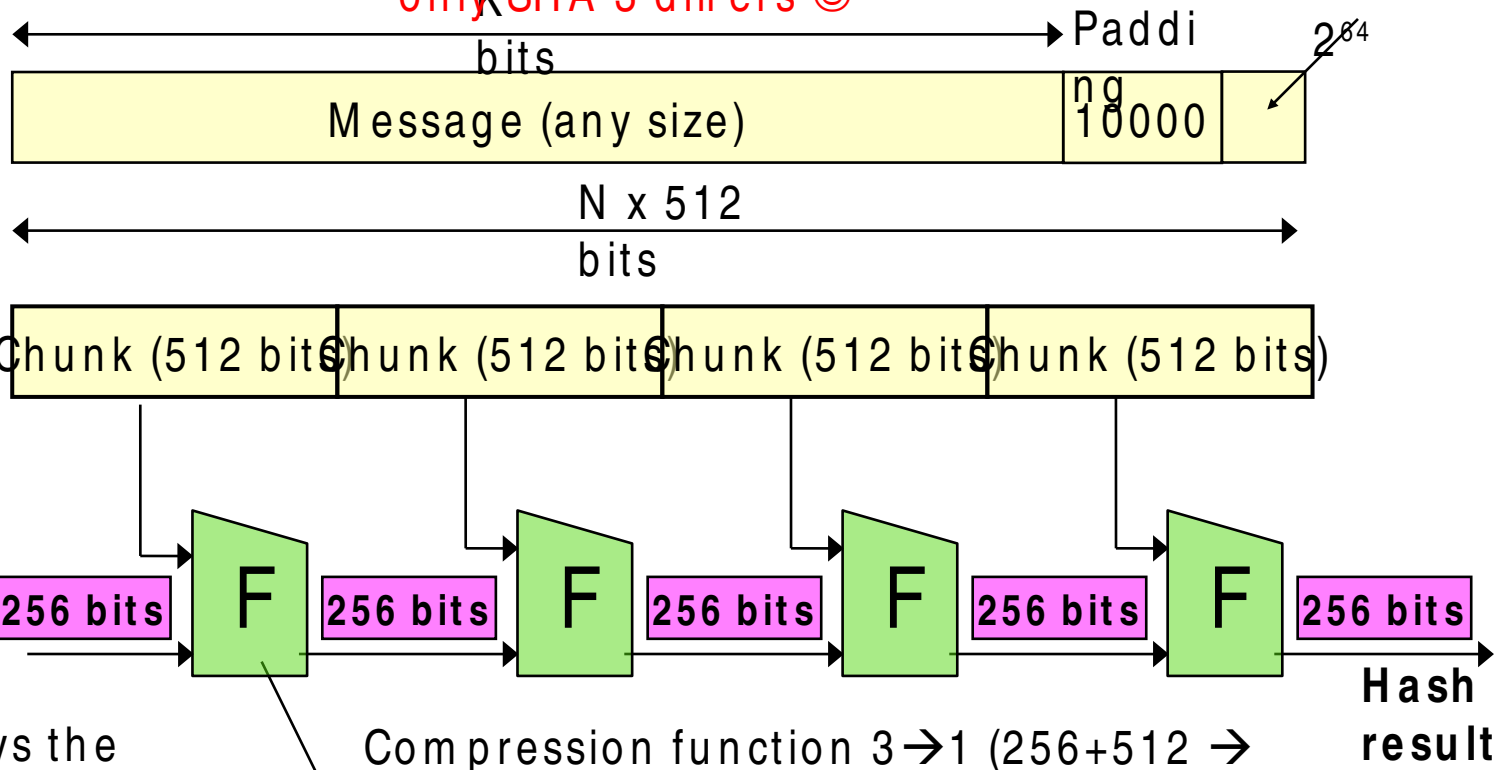
9b05688c

same!

For same M, H(M) must not

change! Giuseppe Bianchi

$H_0^{(6)}$:



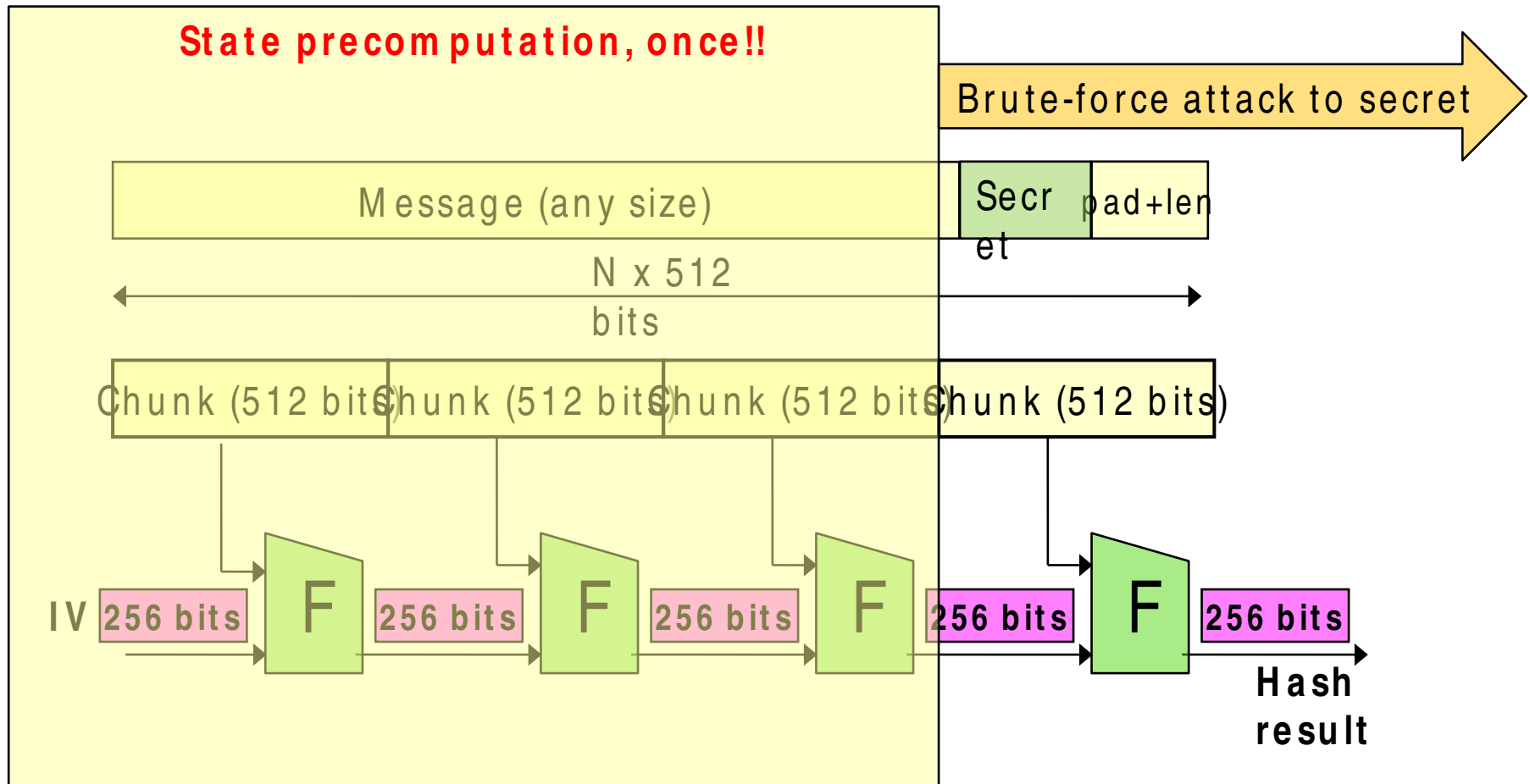
Compression function $3 \rightarrow 1$ ($256 + 512 \rightarrow 256$)

The «heart» of the specific hash function

used

Merkle-Damgård theorem: if F resistant

Secret Suffix: $H(M,S)$?



Weakened security: from N compression blocks per trial, to just 1!

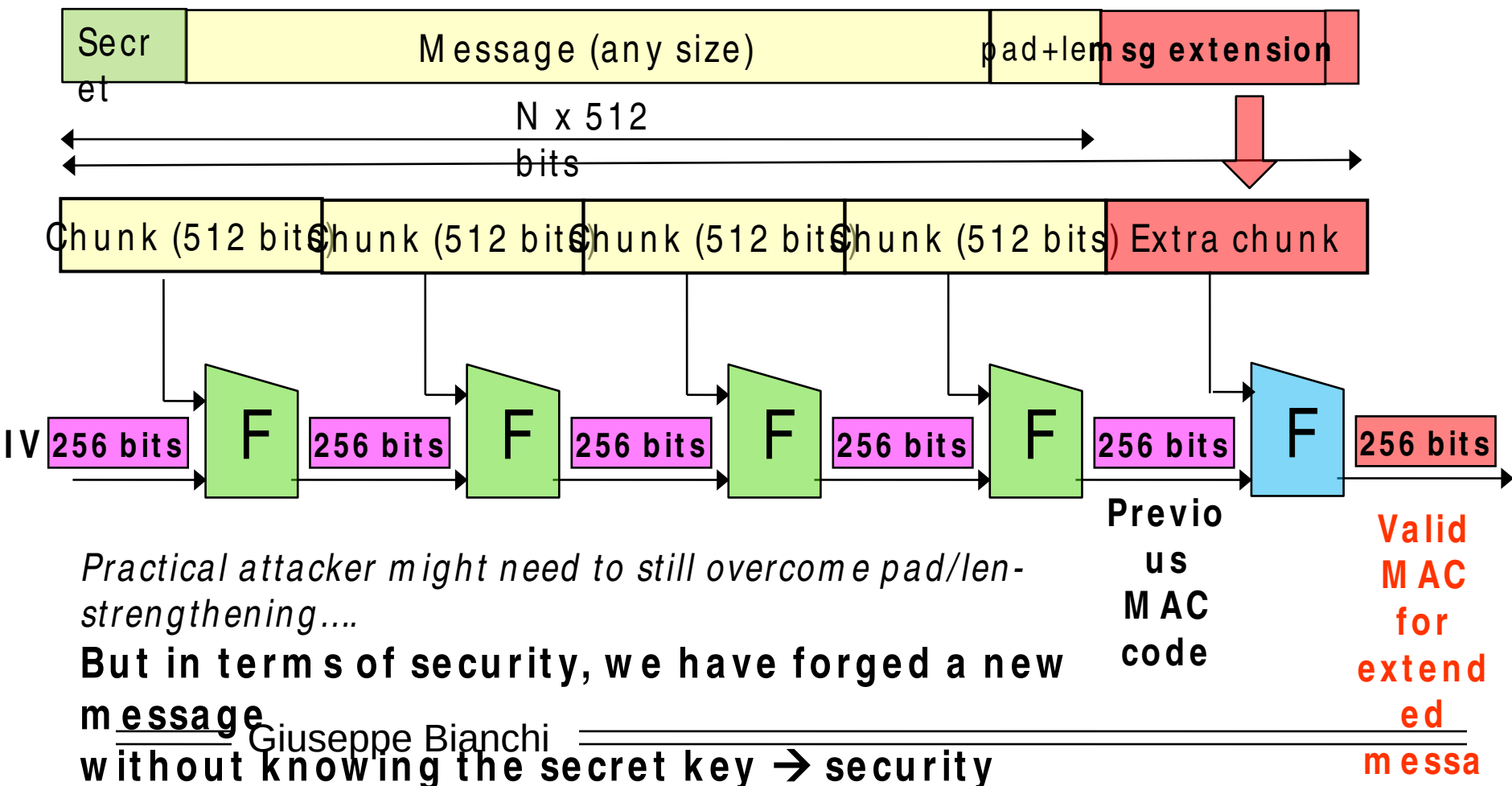
Giuseppe Bianchi

Even worse: collision on msg $\rightarrow$ collision ALSO on

Secret Prefix: $H(S,M)$?

→ Expansion attack!

- ⇒ Message extension does not require knowledge of secret
- Secret already “used” in the (now known) hash state



Expansion attack!

→ **Trivial to “extend” the message!**

→ **Especially critical if secret at the beginning**

⇒ Example: start from $\text{SHA256}(k \parallel x)$, k unknown secret

⇒ Append y

⇒ To compute $\text{SHA256}(k \parallel x \parallel y)$ use iterative Merkle construction!

→ No need to know k !!!

→ **Length (Damgard) strengthening:
helps but does not solve the problem**

⇒ A strong reason to use different constructs

(HMAC)

Giuseppe Bianchi

→ http://csrc.nist.gov/pki/HashWorkshop/2005/Nov1_Presentations/Puniya_hashDesign.pdf

HMAC to the rescue!

→ Lesson learned so far: A secure hash is not enough!

→ Actually, **a secure hash may yield a completely insecure MAC**
(e.g. expansion attack)

→ Question: is there a «secure» way to include a secret in a hash?

- ⇒ Irrespective of the specific hash construction
- ⇒ hash should be considered as a black box!

→ Yes: problem solved in 1996: HMAC

→ Original paper: Bellare, Canetti, Krawczyk, Keying hash functions for message authentication, crypto 96

→ standard: IETF RFC 2104

- ⇒ Provably secure construction
- ⇒ «pluggable» hash (you choose/replace it)
- ⇒ (more or less) same performance of used hash

HMAC construction

$$\text{HMAC}_K(M) = H(K^+ \oplus \text{opad} \parallel H(K^+ \oplus \text{ipad} \parallel M))$$

→ K^+ = shared key, but “extended” to block size

» E.g. SHA256 → padded with zeros up to 512 bits

» *Makes sure secret uses the entire first hash chunk*

128 bit	00000000000000000000000000000000
secret	512 bit Chunk (SHA256 example)

$\oplus$

00110110.00110110.00110110.00110110.
00110110.....

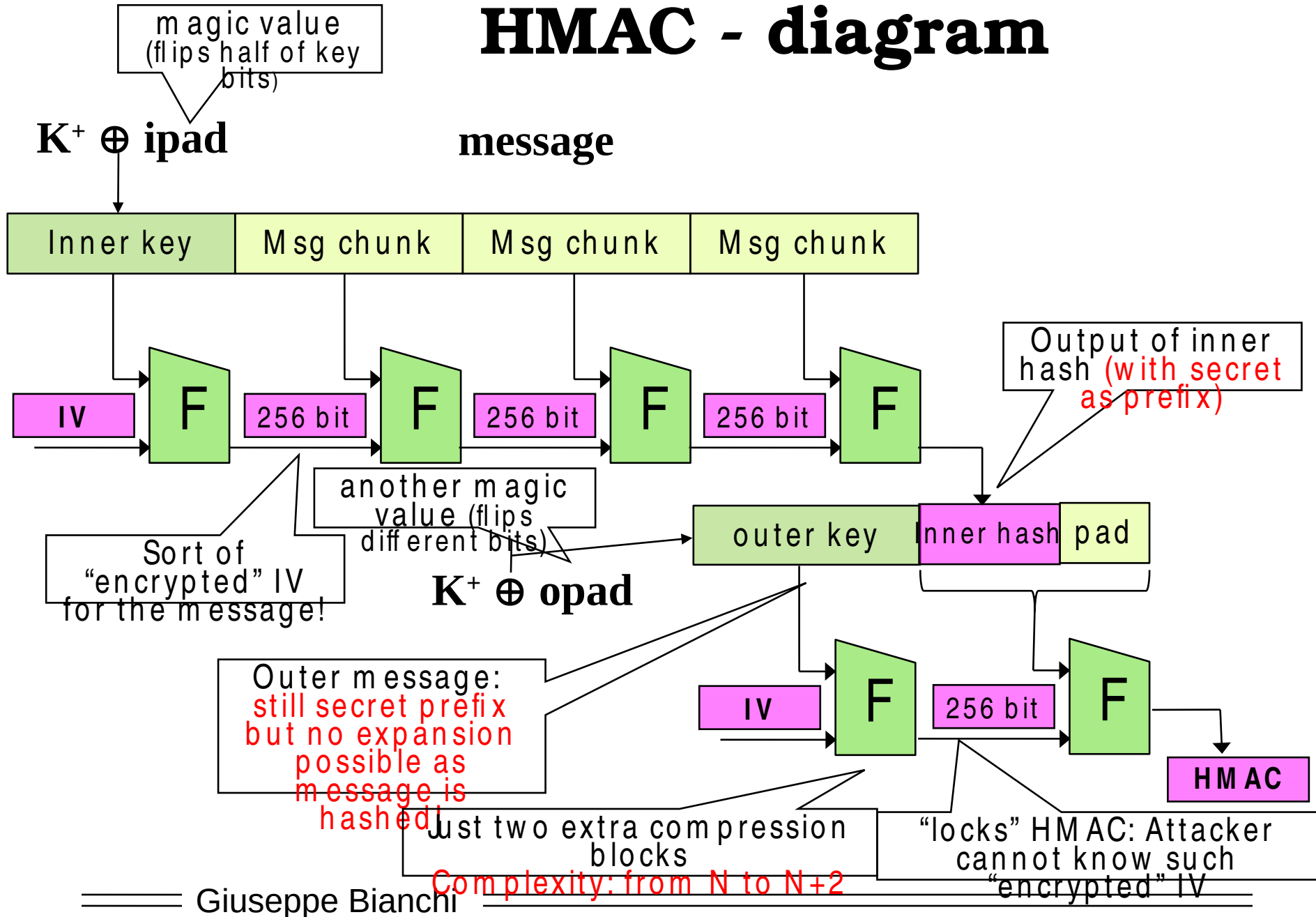
→ opad = 0x36 = 00110110 repeated as needed

→ ipad = 0x5C = 01011100 repeated as needed

» *We would need TWO different (inner and outer) secrets, but since we have only one, let's “pseudo-randomly” derive inner and outer keys from the single key*

(roughly) equivalent to NMAC construction: $H(K_o \parallel H(K_i \parallel M))$

HMAC - diagram



HMAC security

- **quantitatively proven robustness:**
as secure as its underlying hash is
 - ⇒ see Bellare, Canetti, Krawczyk paper for math proof
 - ⇒ **we will not dig into this, further**
- **Actually more secure (surprise!)**
 - ⇒ Bellare 2006: collision resistance NOT necessary
 - Pseudorandomness is the only requirement
 - ⇒ You might even still use HMAC with MD5 or SHA-1 even if they are both broken - there are algorithms to compute collision

Collisions without HMAC

→ **Collision in hash = collision in MAC**

→ **H(M,S) construction:**

⇒ Obvious

→ find collision on first part of the message, then expand

→ **H(S,M) construction:**

⇒ less obvious, but same problem

→ Start from $H(S,x) \rightarrow IV$

→ Find collision $H^*(IV,X1)=H^*(IV,X2)$

→ $M_i = x \parallel \text{pad}(x) \parallel X_i \parallel \text{pad}(X_i)$

→ Hence $H(S,M1)=H(S,M2)$

Take-home

- In crypto, **NEVER** use a function which is «more or less» close to your needs
- Rather, **ONLY** use functions **SPECIFICALLY** designed for your **PRECISE** need!
- **Hash: seemed OK, but was NOT!**
 - ⇒ We needed to create a new function, HMAC

Summary

- **Confidentiality $\neq$ integrity**
- **Message authentication (symmetric key)**
 - ⇒ And formal definition of a secure MAC scheme
- **Replay attacks → MAC not enough → nonces**
- **Ideal vs real world hash functions:**
 - ⇒ Perfect hash: Random Oracle
 - ⇒ Real world hash: Iterative (merkle-damgard) constructions
- **How to include key in hash? Non trivial!**
 - ⇒ Solution: HMAC construction